Introduction to Java 8

02 March 2026 11:20

Java 8 is one of the most important versions of Java released in 2014.
Main Goal:

- Reduce boilerplate code
- Support Functional Programming
- Improve performance
- Easier collection handling

Major Features:

- Lambda Expressions
- Functional Interfaces
- Stream API
- Default & Static Methods in Interface
- Method References
- Optional Class
- New Date and Time API
- Nashorn JavaScript Engine
- Parallel Streams

2. Lambda Expression

Definition
Lambda Expression is an anonymous function (function without name).
Used to implement Functional Interface.
Syntax:

```
(parameters) -> { body }
```

Example Without Lambda (Old Java)

```
interface MyInterface {
   void show();
}

class Test {
   public static void main(String[] args) {

      MyInterface obj = new MyInterface() {
         public void show() {
            System.out.println("Hello");
         }
      };
```

```
        obj.show();
    }
}
```

Example With Lambda (Java 8)

```
interface MyInterface {
    void show();
}

public class Test {
    public static void main(String[] args) {

        MyInterface obj = () -> System.out.println("Hello");

        obj.show();
    }
}
```

Lambda with Parameters

```
interface Add {
    int sum(int a, int b);
}

public class Demo {
    public static void main(String[] args) {

        Add add = (a, b) -> a + b;

        System.out.println(add.sum(10,20));
    }
}
```

3. Functional Interface

Definition
Interface having exactly ONE abstract method.
Annotation:

@FunctionalInterface
Example:

```
@FunctionalInterface
interface Calculator {
   int calculate(int a, int b);
}
```

Valid:

```
interface A {
   void show();
   default void test(){}
}
```

Invalid:

```
interface A {
   void show();
   void run();   // error
}
```

Built-in Functional Interfaces

Predicate

Takes input, returns boolean.

```
Predicate<Integer> p = x -> x > 10;
System.out.println(p.test(20));
```

Function

Takes input, returns output.

```
Function<Integer, Integer> f = x -> x * x;
System.out.println(f.apply(5));
```

Consumer

Takes input, no return.

```
Consumer<String> c = s -> System.out.println(s);
c.accept("Java");
```

Supplier

No input, returns value.

```
Supplier<Double> s = () -> Math.random();
System.out.println(s.get());
```

4. Method Reference

Shortcut of Lambda.
Syntax:

ClassName::methodName

Static Method Reference

```
class Demo {
   static void display() {
      System.out.println("Hello");
   }
}

public class Test {
   public static void main(String[] args) {

      Runnable r = Demo::display;
      r.run();
   }
}
```

Instance Method Reference

```
class A {
   void show() {
      System.out.println("Instance method");
   }
}

A obj = new A();
Runnable r = obj::show;
r.run();
```

5. Stream API

Definition
Stream is used to process collections in functional style.
Advantages:

- Less code
- Parallel processing
- No modification of original data

Create Stream

```
List<Integer> list = Arrays.asList(1,2,3,4,5);

list.stream();
```

Filter Example

```
list.stream()
   .filter(x -> x % 2 == 0)
   .forEach(System.out::println);
```

Map Example

```
list.stream()
   .map(x -> x * x)
   .forEach(System.out::println);
```

Sorted

```
list.stream()
   .sorted()
   .forEach(System.out::println);
```

Collect

```
List<Integer> result =
   list.stream()
      .filter(x -> x > 2)
      .collect(Collectors.toList());
```

Count

```
long count =
   list.stream()
      .filter(x -> x > 3)
      .count();
```

Parallel Stream

```
list.parallelStream()
   .forEach(System.out::println);
```

6. forEach Method

```
list.forEach(x -> System.out.println(x));
```

Using Method Reference:

```
list.forEach(System.out::println);
```

7. Default Methods in Interface

Before Java 8:
Interfaces allowed only abstract methods.
Java 8 allows method body.

```
interface A {

   default void show() {
      System.out.println("Default Method");
   }
}
```

8. Static Methods in Interface

```
interface A {

   static void display() {
      System.out.println("Static Method");
   }
}

A.display();
```

9. Optional Class

Used to avoid NullPointerException.

Without Optional

```
String name = null;
System.out.println(name.length()); // exception
```

With Optional

```
Optional<String> name = Optional.ofNullable(null);

System.out.println(name.orElse("Default"));
```

Important Methods

isPresent()
get()
orElse()
orElseGet()
orElseThrow()
Example:

```
Optional<String> op = Optional.of("Java");

op.ifPresent(System.out::println);
```

10. New Date and Time API

Package:
java.time

Current Date

```
LocalDate date = LocalDate.now();
System.out.println(date);
```

Current Time

```
LocalTime time = LocalTime.now();
System.out.println(time);
```

Date and Time

```
LocalDateTime dt = LocalDateTime.now();
System.out.println(dt);
```

Specific Date

```
LocalDate d = LocalDate.of(2026,3,2);
```

11. Comparator Using Lambda

Old Way:

```
Collections.sort(list, new Comparator<Integer>() {
   public int compare(Integer a, Integer b) {
      return a - b;
   }
});
```

Java 8:

```
Collections.sort(list, (a,b) -> a - b);
```

12. Collectors Class

Used with Stream API.

```
list.stream()
   .collect(Collectors.toSet());
```

Common Methods:

- toList()
- toSet()
- joining()
- counting()
- groupingBy()

Example:

```
Map<Integer, List<String>> map =
list.stream()
   .collect(Collectors.groupingBy(String::length));
```

13. Difference: Collection vs Stream

Collection:

- Stores data
- Can modify data

Stream:

- Processes data
- Does not store
- One-time use

14. Interview Important Points

- Lambda works only with Functional Interface
- Stream does not modify original collection
- Optional prevents NullPointerException
- Default methods solve backward compatibility
- Streams support lazy evaluation
- Parallel stream improves performance

15. Complete Real Example (Best Practice)

```
import java.util.*;
import java.util.stream.*;

public class Demo {

   public static void main(String[] args) {

      List<String> names =
         Arrays.asList("Aman","Rohit","Karan","Ankit");

      List<String> result =
         names.stream()
            .filter(n -> n.startsWith("A"))
            .map(String::toUpperCase)
            .sorted()
            .collect(Collectors.toList());

      result.forEach(System.out::println);
   }
}
```